Homework 7: Dynamic programming on trees

Guideline

Homework chatbot. Open the homework Gem here. Use this Gem when working through the AI-related sections. The bot is already curated with the right answers and hints to provide the right level of guidance to help you learn.

Showing your honest attempt can only help. There is no advantage in cheating. I give 100% credit on attempts, even if they are incorrect. Cheating on homework will also only hurt you. If you cheat, I won't be able to give you the proper feedback to help you learn, nor adjust the difficulty of the quizzes. You will also likely fail on the in-person mastery quizzes, which test concepts tackled in the homework.

Collaboration: Only after solo attempt. You may collaborate and use external resources, but (1) you are encouraged to do the homework **alone first** and (2) you must **attribute** these external resources when you do use them.

AI: Follow guideline. The problems will be very clear on when you are allowed to use AI. Please follow them. Here is why: (1) Unregulated usage of AI harms learning (2) Even in the age of AI, your AI-independent ability matters. The **only** AI tool you should use on this homework is the homework-specific Gemini Gem that I have created for you.

Where AI is allowed on this homework. Every problem has two AI-check steps, and every part **before** an AI-check step is solo. Work out the subproblem, the recurrence and the code on your own first, then bring what you already wrote to the Gem. The whole point of the AI steps is to check work you have already done, not to produce it.

Submission. Upload all files to the Homework 7 assignment on Canvas. **9 files total:**

1. **One answer document** containing your answers to every question below. Name it `lastname-hw7.pdf` (or `.txt`). LaTeX isn't required -- plain text, Google Docs → Download as PDF, or Word → Save as PDF are all fine.
2. **Eight AI chat transcript PDFs**, one for each AI-check step. Start a **new chat** in the Gem for each step, so that each transcript covers only that step:
   - `robber-b.pdf`, `robber-d.pdf`
   - `maxpath-b.pdf`, `maxpath-d.pdf`
   - `score-b.pdf`, `score-d.pdf`
   - `fuel-b.pdf`, `fuel-d.pdf`

Accepted file formats: `.pdf` for everything; `.txt` is also fine for the answer document. Do **not** paste chat content into your answer document, and do **not** submit a link to a Google Doc -- upload the file itself.

Due: Sunday, November 15, 2026 at 11:59pm. See the syllabus for the late-day policy.

Points: 28 total. Remember, I give full points to every question that was attempted, and 0 only for a question left blank. What matters is that you have **thought** through the problems enough to prepare yourself for the quizzes.

1. House robber III (8 pts total)

You are a thief robbing houses arranged in a **binary tree**. Each node holds an amount of money.

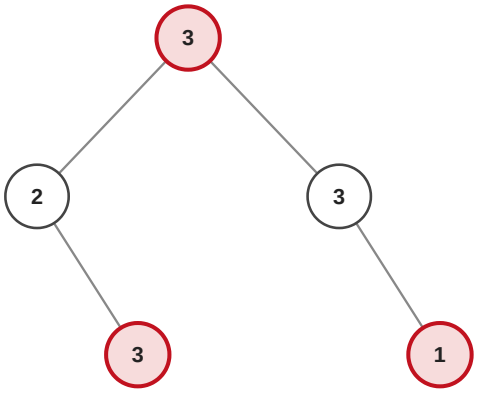
The only rule: you cannot rob two houses that are **directly linked**. That is, if you rob a node, you may not rob either of its children.

Return the **maximum** amount of money you can rob.

Leetcode link

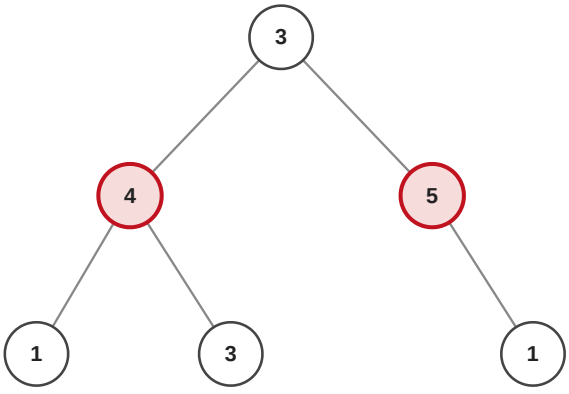
Here are two examples. **Red = robbed.**

Example 1. The answer is 7.



Rob the root and both grandchildren: `3 + 3 + 1 = 7`. You cannot also take the `2` or the middle `3`, because each is a direct child of a house you already robbed.

Example 2. The answer is 9.



Rob both children of the root: `4 + 5 = 9`. Robbing the root instead would block both of them, and `3 + 1 + 3 + 1 = 8` is worse.

(a) No-AI-attempt planning

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Define the subproblem (1 pt)

Write down, in **one line each**, what your `dp` function(s) mean. Use the tree form from lecture -- `dp(node) = ...` -- rather than an array index. If you have multiple states, then provide a definition for each one.

Then, explain how that subproblem will help you solve the original problem.

(ii) Find the recurrence, and argue why it is correct (1 pt)

Express your `dp` value(s) at `node` in terms of its children. Show the exhaustive cases you are taking the max over, and give your base case.

Then, in 2 to 5 sentences, argue why the recurrence is correct.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my subproblem definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "Why isn't one `dp` function enough here?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `robber-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Code (1 pt)

Start a stop watch for **30 minutes** and code this on Leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful Leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `robber-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(e) Runtime (1 pt)

What is the big-O runtime of your solution, and why? Say what `n` stands for.

2. Binary tree maximum path sum (7 pts total)

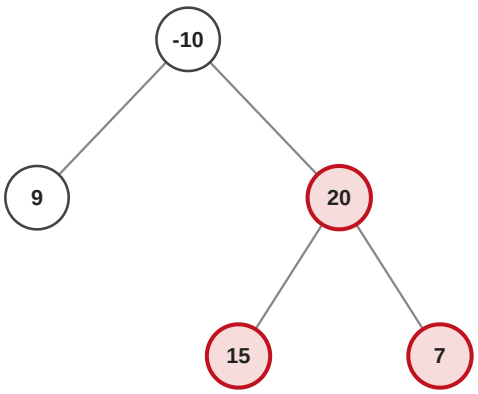
Given the root of a binary tree, return the **maximum path sum** over all non-empty paths. Node values may be negative.

A **path** is a sequence of nodes where every pair of adjacent nodes in the sequence is joined by an edge, and no node appears more than once. A path does **not** have to pass through the root, and it does not have to end at a leaf. The **sum** of a path is the sum of the values of its nodes.

Leetcode link

Here is an example. **Red = the nodes on the best path.**

Example. The answer is 42.



(a) No-AI-attempt planning

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Define the subproblem (1 pt)

Write down, in **one line each**, what your `dp` function(s) mean. Use the tree form from lecture -- `dp(node) = ...` -- rather than an array index.

Then, explain how that subproblem will help you solve the original problem.

(ii) Find the recurrence, and argue why it is correct (1 pt)

Express your `dp` value(s) at `node` in terms of its children. Show the exhaustive cases you are taking the max over, and give your base case.

Then, in 2 to 5 sentences, argue why the recurrence is correct. Be explicit about what your recurrence does when a child's value is negative.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my subproblem definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "The quantity I return to my parent is not the same as the answer I am looking for. Is that expected?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `maxpath-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Code (1 pt)

Start a stop watch for **30 minutes** and code this on Leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful Leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `maxpath-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

3. Count nodes with the highest score (7 pts total)

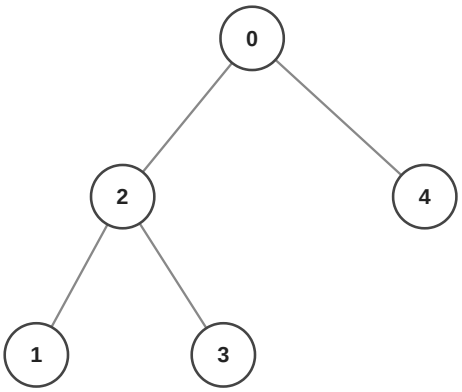
You are given a **binary** tree with `n` nodes labeled `0` to `n - 1`. The tree is described by an array `parents`, where `parents[i]` is the label of node `i`'s parent. Node `0` is the root, so `parents[0] = -1`.

If you **remove** one node from the tree -- the node itself and every edge touching it -- the rest of the tree falls apart into some number of non-empty pieces. The **score** of a node is the **product of the sizes** of the pieces left behind.

Return **how many** nodes have the highest score.

Leetcode link

Here is an example with `parents = [-1, 2, 0, 2, 0]`, so `n = 5`. That array describes this tree. The number inside each node is its **ID**.



Read the array like this. `parents[0] = -1` means node `0` has no parent, so **node `0` is the root**. Then `parents[2] = 0` and `parents[4] = 0` both name `0` as the parent, so **the root's two children are nodes `2` and `4`**. In the same way `parents[1] = 2` and `parents[3] = 2` make nodes `1` and `3` the children of node `2`.

Working out all five scores:

node removed	pieces left behind	score
`0`	`{1, 2, 3}` and `{4}`	`3 x 1 = 3`
`1`	`{0, 2, 3, 4}`	`4`
`2`	`{1}`, `{3}` and `{0, 4}`	`1 x 1 x 2 = 2`
`3`	`{0, 1, 2, 4}`	`4`
`4`	`{0, 1, 2, 3}`	`4`

The highest score is `4`, and **three** nodes achieve it, so the answer is **3**.

(a) No-AI-attempt planning

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Define the subproblem (1 pt)

Write down, in **one line each**, what your `dp` function(s) mean. Use the tree form from lecture -- `dp(node) = ...` -- rather than an array index.

Then, argue how this subproblem will help you solve the original problem. Be explicit about how you get the size of **every** piece a removal leaves behind.

(ii) Find the recurrence, and argue why it is correct (1 pt)

Express your `dp` value(s) at `node` in terms of its children. Give your base case.

Then, in 2 to 5 sentences, argue why the recurrence is correct.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my subproblem definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "I can size some of the pieces but not all of them. Can you give me a hint about the one I am missing?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `score-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Code (1 pt)

Start a stop watch for **30 minutes** and code this on Leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful Leetcode submission.

A note on the input, because we have not done this in class. Problems 1 and 2 hand you a ready-made tree with `node.left` and `node.right`. This one hands you `parents`, a plain array, so build the tree yourself first: maintain a mapping from node ID to its list of children IDs, then as you walk `parents`, add that child to the node's children. Do the conversion before you start the algorithm -- it is much harder to reason about the recurrence while you are still indexing into an array.

A note on the output. Note that you have to return the **number of nodes** that have the highest scores. Overall, you might need two passes. The first pass computes your dp value for each node. The second pass computes both the scores for each node, and the number of nodes that have the highest scores simultaneously.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `score-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

4. Minimum fuel cost to report to the capital (6 pts total)

There are `n` cities numbered `0` to `n - 1`, joined by `n - 1` two-way **roads**. `roads[i] = [a, b]` means there is a road between city `a` and city `b`. The road network forms a **tree**: there is exactly one route between any two cities. City `0` is the **capital**.

Every city other than the capital has exactly one **representative**, who must travel to the capital. There is a car available in every city, and each car seats `seats` people. Driving a car along one road costs **one liter** of fuel.

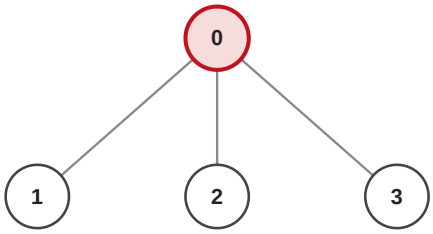
Representatives may **share** cars: when representatives are in the same city, they can continue together in one car, as long as it has room.

Given `roads` and `seats`, return the **minimum** number of liters of fuel needed for everybody to reach the capital.

Leetcode link

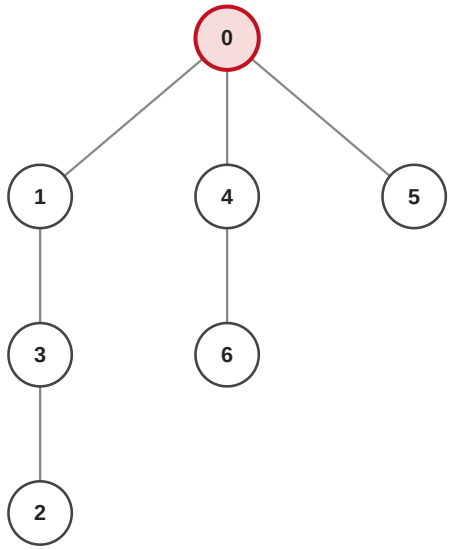
Here are two examples. **Red = the capital.**

Example 1. `roads = [[0,1],[0,2],[0,3]]`, `seats = 5`. The answer is 3.



Each of the three representatives drives one road to the capital: 3 liters. The big cars with 5 seats do not help, because these three only ever meet once they have already arrived.

Example 2. `roads = [[3,1],[3,2],[1,0],[0,4],[0,5],[4,6]]`, `seats = 2`. The answer is 7.



Adding up the liters:

move	people	cars	liters
`2` drives to `3`	1	1	1
`2` and `3` share a car to `1`	2	1	1
`1`, `2` and `3` go to `0`	3	2	2
`6` drives to `4`	1	1	1
`4` and `6` share a car to `0`	2	1	1
`5` drives to `0`	1	1	1
		total	**7**

The step from `1` to `0` is the interesting one: three people do not fit in a car with 2 seats, so that single road costs 2 liters rather than 1.

(a) No-AI-attempt planning (1 pt)

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

Work with these two subproblem definitions

- size(node) = how many people are in this subtree
- min_fuel(node) = min fuel to get everybody from this subtree to meet at this node.

Computing size(node) is easy -- we have already done this many times, so we'll ignore that for our plan.

For min_fuel(node), find the recurrence, and argue why it is correct

That is, express min_fuel(node) in terms of the children's min_fuel and size values.
Hint: If everyone has already gathered to your children, then you just have to compute the additional fuel cost for people to get from the child node to you.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my subproblem definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "How do I know it is never worth sending a car away from the capital?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `fuel-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Code (1 pt)

Start a stop watch for **30 minutes** and code this on Leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful Leetcode submission.

A note on the input. Problems 1 and 2 hand you a ready-made tree, and problem 3 hands you a `parents` array. This one hands you `roads`, a plain list of edges. So build the tree yourself first: make your own adjacency list: map from node ID to a list of neighbor node IDs, fill it in from `roads`, and then traverse it with BFS or DFS to build a separate `children` variable (map from node ID to list of children node IDs), which represents your tree structure. Note that this tree is **not** a binary tree -- a city can have any number of neighbors, so you cannot write `node.left` and `node.right`.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `fuel-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."